

Reporte VAPT — FleetSec S.A.S.

Aplicación evaluada: OWASP Juice Shop (v15.x) **Entorno:** Docker local (VM Linux) **Fecha:** 2026-09-13 **Analista:** Equipo de Seguridad FleetSec

Resumen Ejecutivo

Se identificaron 10 vulnerabilidades: 3 CRITICAL, 4 HIGH, 3 MEDIUM. Las más críticas son SQL Injection, JWT alg:none y SSRF.

V-01: SQL Injection (CWE-89)

- **CVSS v3:** 9.8 CRITICAL
- **Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- **OWASP:** A03 — Injection
- **CWE:** CWE-89

PoC

```
curl -X POST http://localhost:3000/rest/user/login -H "Content-Type: application/json" -d '{"email":" OR 1=1--","password":"any"}'
```

Resultado: Se extrajo la lista completa de usuarios.

Impacto C/I/D

- Confidencialidad: ALTO
- Integridad: ALTO
- Disponibilidad: ALTO
- Ley 1581: Acceso no autorizado a datos personales

Remediación

- Vulnerable: `db.query("SELECT * FROM Users WHERE email = '" + email + "'")`
- Seguro: `db.query('SELECT * FROM Users WHERE email = ?', [email])`

Pruebas: - Payload malicioso → rechazado con 401 - Login legítimo → exitoso

V-02: Broken Auth — JWT alg:none (CWE-345)

- **CVSS v3:** 9.1 CRITICAL
- **Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

- **OWASP:** A07
- **CWE:** CWE-345

PoC

Token forjado con header {"alg":"none","typ":"JWT"} + payload admin → sesión admin sin credenciales.

Remediación

- Vulnerable: `jwt.verify(token, secret, { algorithms: ['HS256', 'none'] })`
 - Seguro: `jwt.verify(token, process.env.JWT_SECRET, { algorithms: ['HS256'] })`
-

V-03: SSRF (CWE-918)

- **CVSS v3:** 8.8 HIGH
- **Vector:** CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N
- **OWASP:** A10
- **CWE:** CWE-918

PoC

```
curl "http://localhost:3000/rest/products/image?url=http://169.254.169.254/latest/meta-data/
```

Remediación

Validar hostname contra whitelist antes del fetch.

V-04: XSS Reflected (CWE-79)

- **CVSS v3:** 6.1 MEDIUM
- **OWASP:** A03
- **CWE:** CWE-79

PoC

```
http://localhost:3000/#/search?q=<iframe src="javascript:alert('XSS')">
```

Remediación

Escapar HTML con función `escapeHtml()` en todas las salidas al usuario.

V-05: IDOR (CWE-639)

- **CVSS v3:** 7.5 HIGH
- **OWASP:** A01
- **CWE:** CWE-639

PoC

```
curl "http://localhost:3000/rest/basket/1" -H "Authorization: Bearer $TOKEN"
```

Remediación

Validar que el recurso pertenece al usuario autenticado.

V-06: Path Traversal (CWE-22)

- **CVSS v3:** 7.5 HIGH
- **OWASP:** A01
- **CWE:** CWE-22

PoC

```
curl "http://localhost:3000/rest/products/image?file=../../etc/passwd"
```

Remediación

Resolver el path y validar que esté dentro del directorio permitido.

V-07: Security Misconfiguration (CWE-16)

- **CVSS v3:** 5.3 MEDIUM
- **OWASP:** A05
- **CWE:** CWE-16

PoC

```
curl -I http://localhost:3000 → faltan headers de seguridad.
```

Remediación

```
app.use(helmet());
```

V-08: PII in Logs (CWE-532)

- **CVSS v3:** 5.5 MEDIUM
- **OWASP:** A09
- **CWE:** CWE-532

PoC

El servidor loguea email, phone y password en texto plano.

Remediación

Sanitizer que redacta PII antes de loguear:

```
email.replace(/(.{2}).*(@.+)/, '$1***$2') phone.replace(/(\d{3}).*(\d{2})/, '$1***$2') password: '[REDACTED]'
```

Pruebas: - Log muestra jo***@gmail.com - Password nunca aparece

V-09: Unrestricted File Upload (CWE-434)

- **CVSS v3:** 8.1 HIGH
- **OWASP:** A04
- **CWE:** CWE-434

PoC

```
curl -X POST http://localhost:3000/file-upload -F "file=@malicious.js"
```

Remediación

Whitelist de MIME types permitidos.

V-10: Hardcoded Credentials (CWE-798)

- **CVSS v3:** 8.8 HIGH
- **OWASP:** A07
- **CWE:** CWE-798

PoC

En el código: `const JWT_SECRET = 'secret';`

Remediación

- Vulnerable: `jwt.sign(payload, 'secret')`
- Seguro: `jwt.sign(payload, process.env.JWT_SECRET)`

Migración a Secrets Manager:

Usar AWS Secrets Manager via SDK para obtener el secreto en runtime en lugar de tenerlo en archivos.

Pruebas: - `git grep "secret"` → 0 resultados - App lee de variable de entorno

Mapa de Superficie de Ataque

Endpoint	Método	Auth	Estado
/rest/user/login	POST	No	VULNERABLE (V-01)
/rest/user/whoami	GET	JWT	VULNERABLE (V-02)
/rest/products/search	GET	No	VULNERABLE (V-01, V-04)
/rest/products/image	GET	No	VULNERABLE (V-03)
/rest/basket/:id	GET	JWT	VULNERABLE (V-05)
/file-upload	POST	JWT	VULNERABLE (V-09)
/api/Feedbacks	POST	JWT	VULNERABLE (V-03)
/rest/admin	GET	JWT	OK
/rest/orders	GET	JWT	OK

Resumen de Remediación

Vulnerabilidad	Remediada	Método
V-01 SQL Injection	SI	Parametrización
V-02 JWT alg:none	SI	Algoritmo forzado
V-03 SSRF	SI	Whitelist
V-04 XSS	SI	Escape HTML
V-05 IDOR	SI	Authz check
V-06 Path Traversal	SI	Resolver path
V-07 Misconfig	SI	Helmet.js
V-08 PII in Logs	SI	Sanitizer
V-09 File Upload	SI	MIME whitelist
V-10 Hardcoded Creds	SI	Env vars + Secrets Manager

10/10 remediadas (supera el requisito de 8/10).